

Final Project Report

Predictive Maintenance of Industrial Equipment Using Machine Learning

1. Introduction

Unexpected machine failures can cause downtime, repair costs, production delays, and safety risks in industrial systems. Predictive maintenance uses machine learning to identify possible failures before they occur, allowing maintenance to be planned instead of reacting after a breakdown.

For this project, the **AI4I 2020 Predictive Maintenance Dataset** from the UCI Machine Learning Repository was utilized. The dataset contains **10,000 machine operation records**. Each record describes the operating condition of a machine using the following input features:

- **Type**: categorical machine/product type
- **Air temperature [K]**: surrounding air temperature
- **Process temperature [K]**: machine/process operating temperature
- **Rotational speed [rpm]**: shaft or motor speed
- **Torque [Nm]**: mechanical load on the machine
- **Tool wear [min]**: accumulated tool wear time

The target variable used in this project is **Machine failure**, which is a binary classification label:

- 0 = no machine failure
- 1 = machine failure

The dataset is highly imbalanced. Out of **10,000 total samples**, only **339 samples** represent machine failure, while **9,661 samples** represent normal operation. This means failure cases make up only **3.39%** of the dataset. Because of this imbalance, accuracy alone is not a reliable metric. A model could predict “no failure” most of the time and still appear accurate while failing to detect actual breakdowns.

This project compares several machine learning models, including **Logistic Regression, Decision Tree, Random Forest, MLP, and XGBoost**, to determine which model performs best for predicting machine failure. Since missing a real failure can be more costly than a false alarm, the project focuses heavily on **recall, precision, F1-score, ROC-AUC, confusion matrices, and threshold tuning** rather than accuracy alone.

The project also investigates threshold tuning from **0.1 to 0.9** to study the trade-off between detecting more failures and reducing false alarms. For the final findings, two representative thresholds were emphasized: **0.1** for maximum failure detection and **0.3** for a more balanced precision-recall trade-off.

2. Methodology

2.1. Dataset Preparation

The dataset was first loaded from the UCI Machine Learning Repository. The input matrix x contained the six machine operation features:

Type,
Air temperature,
Process temperature,
Rotational speed,
Torque,
Tool wear

The output label y was selected as:

Machine failure

Only the binary machine failure target was used for the main classification task. Other failure-type targets such as TWF, HDF, PWF, OSF, and RNF were not used as the main target because the goal of this project was to predict whether a machine fails or not, rather than diagnose the exact type of failure.

```
{'uci_id': 601, 'name': 'AI4I 2020 Predictive Maintenance Dataset', 'repository_url': 'https://archive.ics.uci.edu/dataset/601/ai4i+2020+'}
```

	name	role	type	demographic	description	units	\
0	UID	ID	Integer	None	None	None	
1	Product ID	ID	Categorical	None	None	None	
2	Type	Feature	Categorical	None	None	None	
3	Air temperature	Feature	Continuous	None	None	K	
4	Process temperature	Feature	Continuous	None	None	K	
5	Rotational speed	Feature	Integer	None	None	rpm	
6	Torque	Feature	Continuous	None	None	Nm	
7	Tool wear	Feature	Integer	None	None	min	
8	Machine failure	Target	Integer	None	None	None	
9	TWF	Target	Integer	None	None	None	
10	HDF	Target	Integer	None	None	None	
11	PWF	Target	Integer	None	None	None	
12	OSF	Target	Integer	None	None	None	
13	RNF	Target	Integer	None	None	None	

2.2. Data Preprocessing:

The categorical feature Type was converted into numerical form using one-hot encoding with `pd.get_dummies()`. This was necessary because machine learning models require numerical input. The dataset was split into three subsets:

Training set: used to train the model

Validation set: used to tune hyperparameters

Test set: used for final performance evaluation

The full dataset contained **10,000 samples**. Since the test size was 20%, the final held-out test set contained **2,000 samples**, including 1932 normal samples and 68 failure samples.

Feature scaling was applied using StandardScaler, especially for models sensitive to feature magnitude such as Logistic Regression and MLP. Scaling was fitted only on the training data and then applied to validation and test data to avoid data leakage.

2.3. Models Evaluated:

The following models were trained and compared:

Logistic Regression

Logistic Regression was used as a baseline linear model. Since the dataset was highly imbalanced, class weight="balanced" was used. This caused the model to penalize mistakes in the minority failure class more heavily. The regularization parameter C was tuned using validation data.

The values tested included: 0.01, 0.1, 0.3, 0.5, 0.7, 0.9, 1, 10, 100.

The parameter C is the inverse of the regularization strength. A larger C means weaker regularization, while a smaller C means stronger regularization. The best validation result occurred around: $C = 10$. Although Logistic Regression achieved high recall at some thresholds, it had very low precision, meaning it generated many false alarms.

Decision Trees

A Decision Tree classifier was used to capture nonlinear relationships between the input features and machine failure.

The main hyperparameter tuned was: max_depth. Different depth values were tested using the validation set.

The best depth found was: max_depth = 14.

The Decision Tree improved precision compared with Logistic Regression, but it had lower recall.

Scikit-learn MLPClassifier

A Scikit-learn MLPClassifier was used as one neural network approach. Different hidden layer configurations, regularization values, and learning rates were tested using validation data. This model was useful because it provided a quick neural network baseline using the Scikit-learn interface.

However, Scikit-learn's MLPClassifier provides less direct control over the training process compared with PyTorch.

Custom PyTorch MLP

A second MLP was implemented using PyTorch. The PyTorch implementation allowed direct control over the training loop, including:

- Forward propagation
- Loss calculation
- Backpropagation
- Optimizer updates
- Training loss tracking
- Validation loss tracking

The PyTorch MLP used binary classification with BCEWithLogitsLoss. Training and validation loss were plotted over epochs to show the learning process.

Using both Scikit-learn MLPClassifier and PyTorch MLP helped compare a high-level neural network implementation with a custom deep learning implementation. The PyTorch version allowed more control and visibility into training behavior.

Random Forest

Random Forest was used as an ensemble method that combines multiple decision trees. This reduces the instability and overfitting risk of a single Decision Tree.

The tuned hyperparameters included:

n_estimators

max_depth

Random Forest performed strongly, especially when the threshold was adjusted. It provided high recall at lower thresholds, making it useful when the priority is minimizing missed failures.

XGBoost

XGBoost was used as the strongest boosted tree model. The tuned hyperparameters included:

n_estimators, max_depth, learning_rate, subsample, colsample_bytree

The best validation configuration was approximately:

n_estimators = 200

max_depth = 5

learning_rate = 0.2

XGBoost builds trees sequentially. Each new tree attempts to correct the errors made by previous trees. This makes it powerful for tabular datasets and useful for identifying difficult minority-class samples.

2.4.Threshold Tuning:

The default binary classification threshold is usually: 0.5.

However, for predictive maintenance, the default threshold may not be ideal. Missing a real failure may be more costly than producing a false alarm. Therefore, thresholds from **0.1 to 0.9** were tested to study the trade-off between recall and precision.

For final reporting, two thresholds were selected:

Threshold = 0.1: Failure-Sensitive Mode

This setting makes the model more likely to predict failure. It increases recall, meaning more failures are detected, but it also increases false positives.

This threshold is useful when:

- missed failures are very costly
- safety is important
- false alarms are acceptable

Threshold = 0.3: Balanced Mode

This setting reduces false positives compared with threshold 0.1 while still detecting many failures.

This threshold is useful when:

- unnecessary maintenance is costly
- both precision and recall matter
- a balanced trade-off is preferred

Threshold tuning was a key part of the project because it showed that model behavior depends not only on the algorithm, but also on the chosen decision boundary.

3. Results

3.1. Final Comparison at Threshold = 0.1

	Model	Precision (Failure)	Recall (Failure)	F1 (Failure)	ROC-AUC
5	XGBoost	0.527273	0.852941	0.651685	0.974516
3	MLP (PyTorch)	0.509091	0.823529	0.629213	0.963547
1	Decision Tree	0.586667	0.647059	0.615385	0.816511
2	MLP (Sklearn)	0.363636	0.823529	0.504505	0.969013
4	Random Forest	0.287736	0.897059	0.435714	0.964716
0	Logistic Regression	0.054783	0.985294	0.103796	0.903711

Interpretation

At threshold 0.1, Logistic Regression achieved the highest recall at 0.985, but its precision was extremely low at 0.055. This means it detected almost all failures, but it also generated many false alarms. Random Forest also achieved high recall, but its precision remained low.

XGBoost provided the best overall performance at this threshold. It achieved strong recall of 0.853 while maintaining much better precision than Logistic Regression and Random Forest. It also achieved the highest F1-score and ROC-AUC. This makes XGBoost the strongest failure-sensitive model.

3.2. Final Comparison at Threshold = 0.3

	Model	Precision (Failure)	Recall (Failure)	F1 (Failure)	ROC-AUC
5	XGBoost	0.800000	0.764706	0.781955	0.974516
3	MLP (PyTorch)	0.676471	0.676471	0.676471	0.963547
4	Random Forest	0.552083	0.779412	0.646341	0.964716
2	MLP (Sklearn)	0.583333	0.720588	0.644737	0.969013
1	Decision Tree	0.586667	0.647059	0.615385	0.816511
0	Logistic Regression	0.092424	0.897059	0.167582	0.903711

Interpretation

At threshold 0.3, XGBoost clearly achieved the best overall balance. It had the highest precision, strong recall, the highest F1-score, and the highest ROC-AUC. Compared with threshold 0.1, increasing the threshold improved precision and reduced false alarms.

Random Forest still achieved strong recall, but its precision was lower than XGBoost. The PyTorch MLP and Scikit-learn MLPClassifier also performed well, but neither matched XGBoost's balance of precision and recall.

Logistic Regression continued to have high recall but extremely low precision, making it less practical for real predictive maintenance use because it would generate too many false alarms.

4. Discussion

4.1. Why not use Accuracy:

Only 3.39% of the dataset represents failure cases; accuracy alone is misleading. A model can achieve high accuracy by predicting normal operation most of the time. However, such a model would be useless for predictive maintenance if it failed to detect actual breakdowns. This is why precision, recall, F1-score, ROC-AUC, and confusion matrices were used. Recall was especially important because it measures how many actual failures were detected.

4.2. Why XGBoost Performed Best:

XGBoost achieved the best overall performance because it is well-suited for structured tabular data. It can capture nonlinear feature interactions using tree-based splits. This is important because machine failure may depend on combinations of features, such as high torque with high temperature or high rotational speed with tool wear. XGBoost also uses boosting, where trees are built sequentially. Each new tree corrects errors made by previous trees. This helps the model focus on difficult samples, including rare failure cases.

At threshold 0.3, XGBoost achieved:

Metric	Value
Precision	0.800
Recall	0.765
F1-score	0.782
ROC-AUC	0.975

4.3. Why XGBoost Outperformed the MLP Models:

Although both MLP models performed well, XGBoost outperformed them because the dataset is structured tabular data. Neural networks often perform best on large, complex, unstructured datasets such as images, audio, and text. In contrast, boosted tree models often perform extremely well on tabular datasets because they naturally learn threshold-based decision rules.

5. Conclusion

This project evaluated multiple machine learning models for predicting machine failure using the AI4I 2020 Predictive Maintenance Dataset. The dataset contained 10,000 samples and was highly imbalanced, with only 339 failure cases. Because of this imbalance, accuracy alone was not sufficient for evaluation. Instead, the project focused on precision, recall, F1-score, ROC-AUC, and confusion matrices. Logistic Regression served as a baseline but produced too many false alarms. The Decision Tree improved interpretability but was limited by overfitting risk. Random Forest performed strongly, especially for detecting failures at adjusted thresholds. Both Scikit-learn MLPClassifier and PyTorch MLP showed that neural networks can learn useful failure patterns, but they did not outperform XGBoost. XGBoost achieved the best overall performance. At threshold 0.3, it reached a precision of 0.800, recall of 0.765, F1-score of 0.782, and ROC-AUC of 0.975. This made it the strongest model for balancing failure detection and false alarm reduction. The results also showed that threshold tuning is critical in predictive maintenance. Lower thresholds increase failure detection but also increase false alarms. Higher thresholds reduce false alarms but may miss more failures. Therefore, the best predictive maintenance system should be selected based on both model performance and real-world operational priorities.